

7.2 ACT 训练（推荐入门）

概述

ACT（Action Chunking with Transformers）是 LeRobot 中最推荐入门的算法，训练稳定、显存需求低（8GB 可用），是大多数 SO-101 场景的首选方案。

算法背景

- 论文：[Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (2023)]

(<https://arxiv.org/abs/2304.13705>)

- 核心创新：将未来一段时间内的动作（Chunk）作为整体预测，并通过 CVAE 编码动作多样性，显著提升精细操作的成功率

训练命令

基础训练命令

Code block

```
1  conda activate lerobot
2  cd ~/lerobot
3
4  lerobot-train \
5      --policy.type=act \
```

```
6 --dataset.repo_id=YOUR_HF_USERNAME/my_dataset \  
7 --training.num_epochs=200 \  
8 --training.batch_size=32 \  
9 --training.save_checkpoint=true \  
10 --output_dir=outputs/train/act_my_task
```

完整训练命令（推荐参数）

Code block

```
1 lerobot-train \  
2 --policy.type=act \  
3 --dataset.repo_id=YOUR_HF_USERNAME/my_dataset \  
4 --dataset.image_transforms.enable=true \  
5 --policy.n_action_steps=100 \  
6 --policy.chunk_size=100 \  
7 --policy.dim_model=512 \  
8 --training.num_epochs=200 \  
9 --training.batch_size=32 \  
10 --training.lr=1e-4 \  
11 --training.save_checkpoint=true \  
12 --training.use_wandb=true \  
13 --output_dir=outputs/train/act_my_task
```

关键参数说明

参数	说明	推荐值
--policy.type	算法类型	act
--dataset.repo_id	训练数据集	用户名/数据集名
--policy.chunk_size	动作序列长度	100（约 3s @ 60fps）

<code>--policy.n_action_steps</code>	推理时执行的动作步数	<code>100</code>
<code>--policy.dim_model</code>	Transformer 隐藏维度	<code>512</code> (小) / <code>1024</code> (大)
<code>--training.num_epochs</code>	训练轮数	<code>200</code> (快速验证) / <code>1000</code> (正式训练)
<code>--training.batch_size</code>	批次大小	<code>32</code> (8GB) / <code>64</code> (24GB)
<code>--training.lr</code>	学习率	<code>1e-4</code>

使用预训练权重微调

ACT 支持从现有检查点继续训练：

Code block

```
1  lerobot-train \  
2      --policy.type=act \  
3      --dataset.repo_id=YOUR_HF_USERNAME/my_dataset \  
4      --training.resume=true \  
5      --output_dir=outputs/train/act_my_task  # 继续之前的训练目录
```

数据增强

加入图像数据增强有助于提升泛化性：

Code block

```
1  lerobot-train \
2      --policy.type=act \
3      --dataset.repo_id=YOUR_HF_USERNAME/my_dataset \
4      --dataset.image_transforms.enable=true \
5      --dataset.image_transforms.max_num_transforms=3 \
6      --dataset.image_transforms.random_brightness.max_delta=0.5 \
7      --dataset.image_transforms.random_contrast.min_max="[0.5, 2.0]"
```

训练监控

使用 wandb 监控训练曲线

Code block

```
1  # 训练中添加 wandb 参数
2  --training.use_wandb=true \
3  --training.wandb_project=my_robot_project \
4  --training.wandb_entity=YOUR_HF_USERNAME
```

在 [wandb.ai](#) 查看：

- `train/loss`：训练损失（应持续下降）
- `eval/success_rate`：评估成功率（如配置了评估环境）

判断训练收敛

指标	良好状态
训练损失	从 >1.0 下降到 <0.1
损失曲线	平滑下降，无明显震荡

训练步数	通常 50,000-200,000 步
------	---------------------

训练时长参考

硬件	数据集规模	参考训练时间
RTX 3090 (24GB)	50 episodes	~2 小时
RTX 3090 (24GB)	200 episodes	~8 小时
RTX 4090 (24GB)	200 episodes	~5 小时
A100 (40GB)	200 episodes	~3 小时

获取模型权重

训练完成后，模型保存在：

Code block

```
1 outputs/train/act_my_task/checkpoints/last/pretrained_model/
```

该目录可直接用于推理（参见 [9-模型推理与部署/2-各模型推理命令汇总.md](../9-模型推理与部署/2-各模型推理命令汇总.md)）。

上传到 HuggingFace

Code block

```
1 huggingface-cli upload YOUR_HF_USERNAME/act_my_task \  
2     outputs/train/act_my_task/checkpoints/last/pretrained_model/
```

常见问题

Q: 训练损失不下降怎么办?

A:

1. 检查数据集质量（通过可视化工具抽查几个 episode）
2. 尝试增大学习率（`1e-3`）或减小（`5e-5`）
3. 检查数据集 episode 数量是否足够（至少 50 个）

Q: 训练时 GPU 显存不足?

A:

- 减小 `batch_size`（从 32 → 16 → 8）
- 减小 `dim_model`（从 512 → 256）
- 减小 `chunk_size`（从 100 → 50）

参考资料

- [ACT 原始论文](#)
- [HuggingFace LeRobot 训练文档](#)